

A Structure-Level Action Language for LLM-Based Coding Agents

Russell Rozenbaum, Cyrus Desai, Andrew Blinn, MS, Cyrus Omar, PhD

Background

- Large Language Models (LLMs) are revolutionizing the frontier of **automated program synthesis**.
- Works such as SWE-Agent² have introduced the concept of the **agent-computer interface (ACI)**, showing that LLM-based coding agents, like humans, need IDEs too.
- Existing ACIs feature action commands that **derive locality within the code base through line indices**. We propose that a **structure-level action language, deriving locality through high-level semantics of the program itself**, enable agents to more effectively understand, navigate, and edit a program during task completion.

Driving Questions

- Is the use of a high, structure-level action language within an LLM-based coding agent tenable? Are its effects favorable, or even discernible, from existing action languages?
- Do LLM-based agents prove to be a useful tool in human learning for lesser-known, low-resource programming languages such as Hazel³?

Agent-Computer Action Language

Category	Command	Documentation
<i>File Viewer</i>	goto_definition <variable_name>	Places the cursor at the nearest definition of the matched variable_name .
	goto_type_definition <variable_name>	Places the cursor at the nearest type definition for the matched variable_name .
	show_references <variable_name>	Displays a list of definitions where the variable matching variable_name is referenced.
	scroll_down	Moves cursor to preceding definition, if one exists.
	scroll_up	Moves cursor to succeeding definition, if one exists.
<i>File Editing</i>	replace <replacement_text>	Replaces the current definition with replacement_text .
	insert <new_text>	Inserts new_text after the definition the cursor is currently at.
	delete	Deletes the definition the cursor is currently at.
<i>Task</i>	submit	Ends the iterative task completion process.

User-Agent Interface

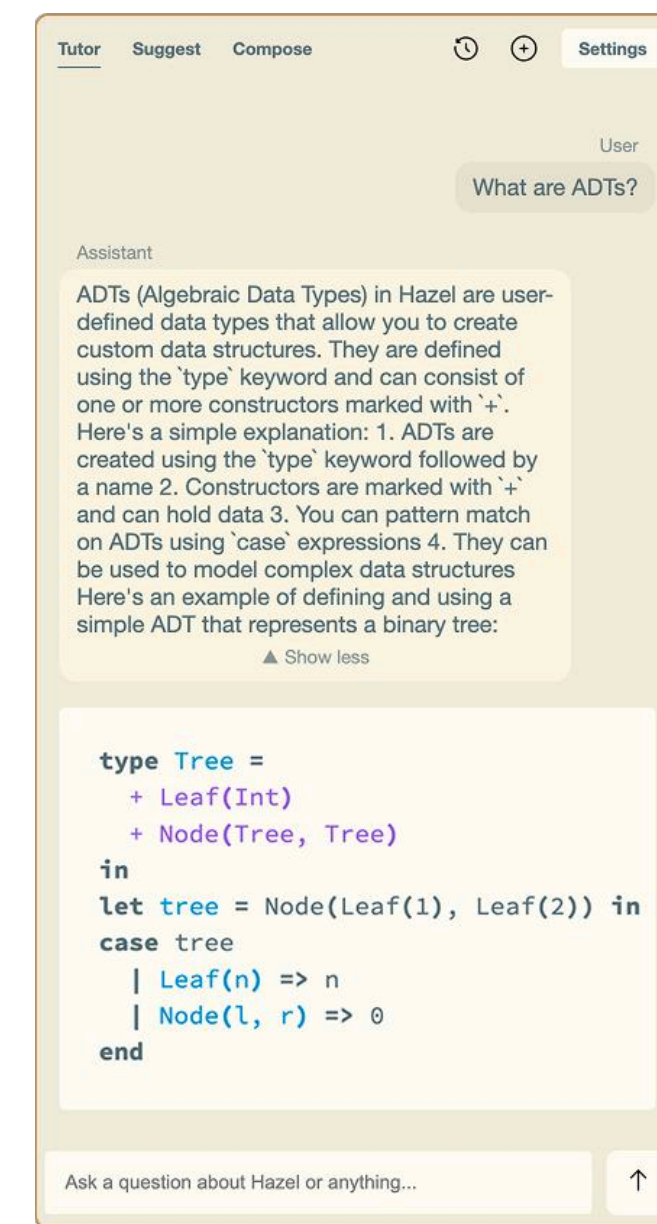


(Left) A user requests the completion of a hole located in the body of the function `emoji_farm`. The LLM responds with a suggested completion, where the user then presses `tab` to accept this completion.

This “Suggest” mode for the agent is an implementation of the work done by Andrew Blinn and team on ChatLSP and the Static Contextualization of LLMs¹ in code completion tasks.

(Right) A user asks the LLM a question regarding ADTs. The LLM delivers a thorough and informative response.

The “Tutor” mode is a simple one-on-one chat between the user and LLM. The LLM is instructed to act as a helpful Hazel³ tutor and is prompted with adequate Hazel³ documentation.



Acknowledgments

Thanks to Professor Omar for guidance, Andrew Blinn for feedback, Cyrus Desai for collaboration, and Professor Chen for reviewing my thesis.

References

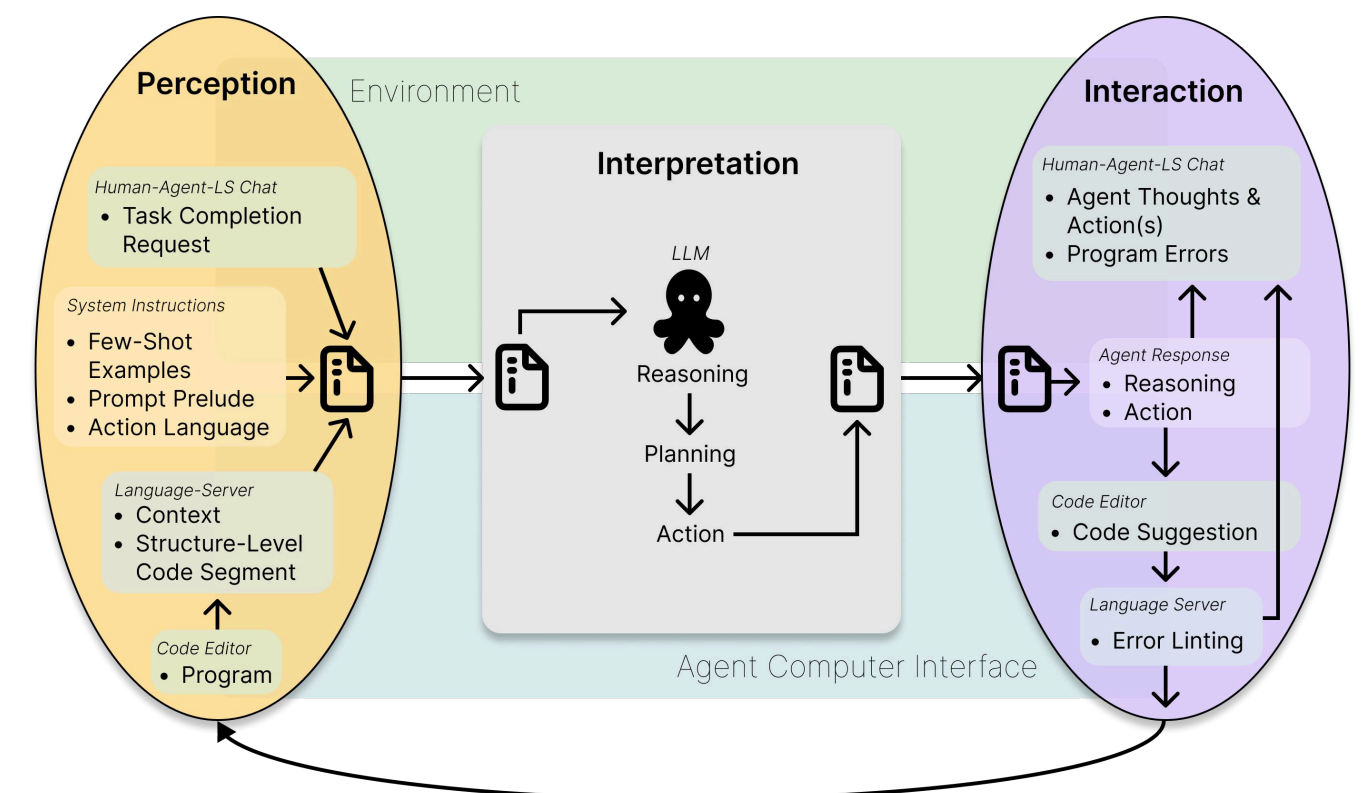
- Blinn, A. et al. Statically Contextualizing Large Language Models with Typed Holes (2024).
- Yang, J. & Jimenez, C. et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering (2024).
- Omar, C. et al. Hazelnut: A Bidirectionally Typed Structure Editor Calculus (2017).

Future Steps

We are currently still in the process of **implementing our proposed agent architecture** and **evaluating our driving questions and goals**. Due to this, both the action language and architecture are subject to change during the experimentation process.

To avoid nullifying our results due to data contamination, we intend to build our own, private **benchmark**, extending the one introduced in the work of ChatLSP¹. We will use this benchmark to evaluate our agent's performance on software engineering tasks in Hazel³.

Agent Architecture



- A diagram of our Agent-Architecture, comprising three main stages: **Perception**, **Interpretation**, and **Interaction**.